EcoQuest

Documentación del proyecto de software

Proyecto Informático II
Escuela Técnica Defensa de Buenos Aires · 5.º 7.ª
Profesor Oscar Obregón

Integrantes: Cuevas Valentina y Missel Betancourt
Entrega: 3 de noviembre de 2026

Resumen

EcoQuest es una aplicación web educativa que ayuda a separar residuos y consultar puntos de recepción de CABA. Combina cuentas de usuario, análisis de fotografías, mapa y actividades de juego. Este documento describe el alcance implementado, sus procesos y datos, y el procedimiento para instalar y probar el prototipo local.

Las cuentas se conservan en MySQL. El juego utiliza SQLite en memoria: XP, escaneos, sesiones, rachas, logros e historial se pierden cuando la API se detiene. El análisis de imágenes utiliza Gemini y requiere una clave propia y conexión a Internet.

Contenido

Sección	Contenido	Página
1	Definición y requisitos	2
2	Instalación desde GitHub y ejecución	3 y 4
3	Arquitectura y módulos	5
4	DER y diccionario de datos	6 y 7
5	Procesos, contexto y DFD	8 a 11
6	Pantallas y reglas del juego	12
7	Especificación de API	13 y 14
8	Pruebas, conclusiones y entrega	15

Repositorio del proyecto: https://github.com/criseri23/ecoquest2

Acceso al prototipo después de iniciar el proyecto: http://localhost:5228/pages/click.html.

1 Definición y requisitos

1.1 Propósito y personas usuarias

El objetivo es orientar la separación de residuos y la elección de un punto compatible, usando actividades que motiven hábitos de reciclaje. Está dirigido a personas que usan un navegador de computadora o teléfono. La instalación requiere conocimientos básicos de archivos, terminal y bases de datos; los pasos se detallan en la sección 2.

El proyecto es original de Cuevas Valentina y Missel Betancourt, según la documentación de autoría. Se desarrolló de forma incremental: pantallas, conexión con la API, cuentas, análisis, mapa y juego. No es una migración de un sistema anterior.

1.2 Requisitos funcionales

Código	Comportamiento implementado
RF01	Registrar nombre de 3 a 24 caracteres, email válido y contraseña de 8 a 128 caracteres. Evitar emails duplicados.
RF02	Iniciar sesión con email o nombre y contraseña; cerrar la sesión.
RF03	Conservar cuentas y hash de contraseña en MySQL.
RF04	Analizar una foto y mostrar material, recomendación y posibles errores.
RF05	Consultar puntos del catálogo de CABA y validar residuos con ubicación y compatibilidad.
RF06	Calcular XP, nivel, misiones, rachas, calendario, historial y ranking temporales.
RF07	Permitir acceso como invitada sin cuenta; no guardar escaneos ni asignar XP en el servidor.
RF08	Mostrar errores de validación y servicios; permitir preferencias locales de movimiento.

1.3 Requisitos no funcionales y límites

Se utiliza navegador web, .NET SDK 10 y MySQL/MariaDB. La interfaz incluye adaptación a distintos tamaños de pantalla. Las contraseñas se verifican mediante hash. La IA y los recursos de mapa pueden requerir Internet; la ubicación necesita permiso y precisión suficiente.

No se implementan recuperación de contraseña, administración, guardado permanente del juego ni cuidados o accesorios de la mascota. La pantalla Mascota está marcada como “Próximamente”. La foto y la cercanía del dispositivo no demuestran por sí solas que se haya depositado físicamente un residuo.

2 Instalación desde GitHub

2.1 Preparar el equipo y descargar

1. Tener instalado .NET SDK 10, un navegador y MySQL o MariaDB. En Windows se puede usar XAMPP para administrar MySQL y phpMyAdmin. Verificar el SDK desde PowerShell con:

```
dotnet --version
```

2. Abrir https://github.com/criseri23/ecoquest2. En GitHub elegir Code y luego Download ZIP. Extraer todo el ZIP; no ejecutar archivos directamente dentro del archivo comprimido.

3. Renombrar la carpeta extraída como EcoQuest para seguir estos ejemplos. El nombre es una comodidad, no una exigencia del programa. Entrar a la carpeta que contiene backend y Frontend; si hay otra carpeta adentro, usar esa carpeta interna como raíz.

```
EcoQuest
  backend
    EcoQuestAPI
    EcoQuest.Datos
    EcoQuest.Logica
  Frontend
```

2.2 Iniciar MySQL y crear la base ecoquest

4. Abrir XAMPP Control Panel y pulsar Start en MySQL. Para administrar la base desde phpMyAdmin, iniciar también Apache y abrir http://localhost/phpmyadmin. Apache no es necesario para servir EcoQuest: la aplicación web la sirve ASP.NET Core.

5. Crear una base de datos llamada exactamente ecoquest, en minúsculas. Desde la pestaña SQL de phpMyAdmin se puede ejecutar:

```
CREATE DATABASE IF NOT EXISTS ecoquest
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

No hace falta crear Usuarios manualmente. El backend crea esa tabla al iniciar si todavía no existe. La base ecoquest sí debe existir antes del arranque.

2.3 Revisar la conexión

6. Abrir backend/EcoQuestAPI/appsettings.json. La clave ConnectionStrings:EcoQuestDb debe apuntar al servidor, puerto, usuario y contraseña del MySQL local. Ejemplo para XAMPP con root sin contraseña:

```
"ConnectionStrings": {
  "EcoQuestDb": "server=127.0.0.1;port=3306;database=ecoquest;user=root;password="
}
```

Si el usuario root tiene contraseña o MySQL usa otro puerto, adaptar esos valores. No copiar contraseñas privadas a un repositorio público.

2 Instalación y ejecución continuada

2.4 Abrir la terminal y ejecutar

7. Abrir PowerShell en la carpeta raíz EcoQuest. Se puede escribir powershell en la barra de direcciones del Explorador de archivos. Ejecutar los comandos por separado:

```
cd backend
cd EcoQuestAPI
dotnet restore
dotnet run --launch-profile http
```

El comando es dotnet, todo junto. El primer arranque puede necesitar Internet para descargar paquetes. El perfil http fija el puerto 5228. Dejar la terminal abierta mientras se usa la aplicación.

También se puede iniciar desde la raíz EcoQuest con un solo comando equivalente:

```
dotnet run --project backend\EcoQuestAPI\EcoQuestAPI.csproj --launch-profile http
```

8. Cuando aparezca “Now listening on: http://localhost:5228”, abrir:

http://localhost:5228/pages/click.html

La pantalla ofrece Iniciar sesión, Registrarse y Entrar como invitada. No abrir los HTML con doble clic: las solicitudes necesitan la API. Para detener el servidor, presionar Ctrl+C en la terminal.

2.5 Configurar Gemini para el análisis de fotografías

La descarga no incluye una clave privada de Gemini. Para habilitar el análisis real, configurar una clave válida en la misma ventana de PowerShell antes de iniciar la API:

```
$env:GEMINI_API_KEY = "PEGAR_TU_CLAVE_PROPIA"
dotnet run --launch-profile http
```

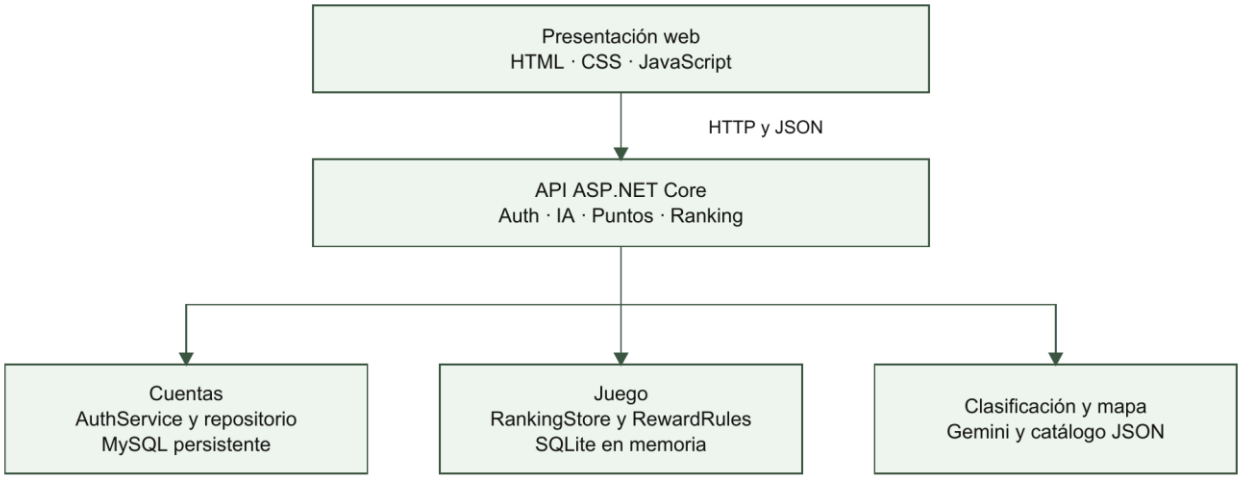
La variable anterior dura durante esa sesión de PowerShell. Se debe reemplazar el texto de ejemplo por una clave propia. Sin clave, la aplicación puede ofrecer sus otras pantallas, pero /api/ia/analizar devuelve un error de configuración; no existe una respuesta simulada automática que sustituya a Gemini.

2.6 Problemas frecuentes

Síntoma	Revisión
dotnet no se reconoce	Instalar el SDK 10 y volver a abrir PowerShell.
No se encuentra el proyecto	Entrar a la carpeta que contiene EcoQuestAPI.csproj.
No conecta con MySQL	Encender MySQL; revisar host, puerto, usuario y contraseña.
Unknown database ecoquest	Crear la base ecoquest antes de ejecutar la API.
Puerto 5228 ocupado	Cerrar la instancia anterior de EcoQuest y volver a iniciar.
IA o ubicación no disponibles	Revisar clave, Internet, servicio Gemini y permisos del navegador.

3 Arquitectura y módulos

La presentación se organiza en Frontend/pages, Frontend/js y Frontend/css. El backend se divide en EcoQuestAPI, EcoQuest.Logica y EcoQuest.Datos. La API sirve los archivos del frontend desde el mismo origen y coordina las solicitudes.



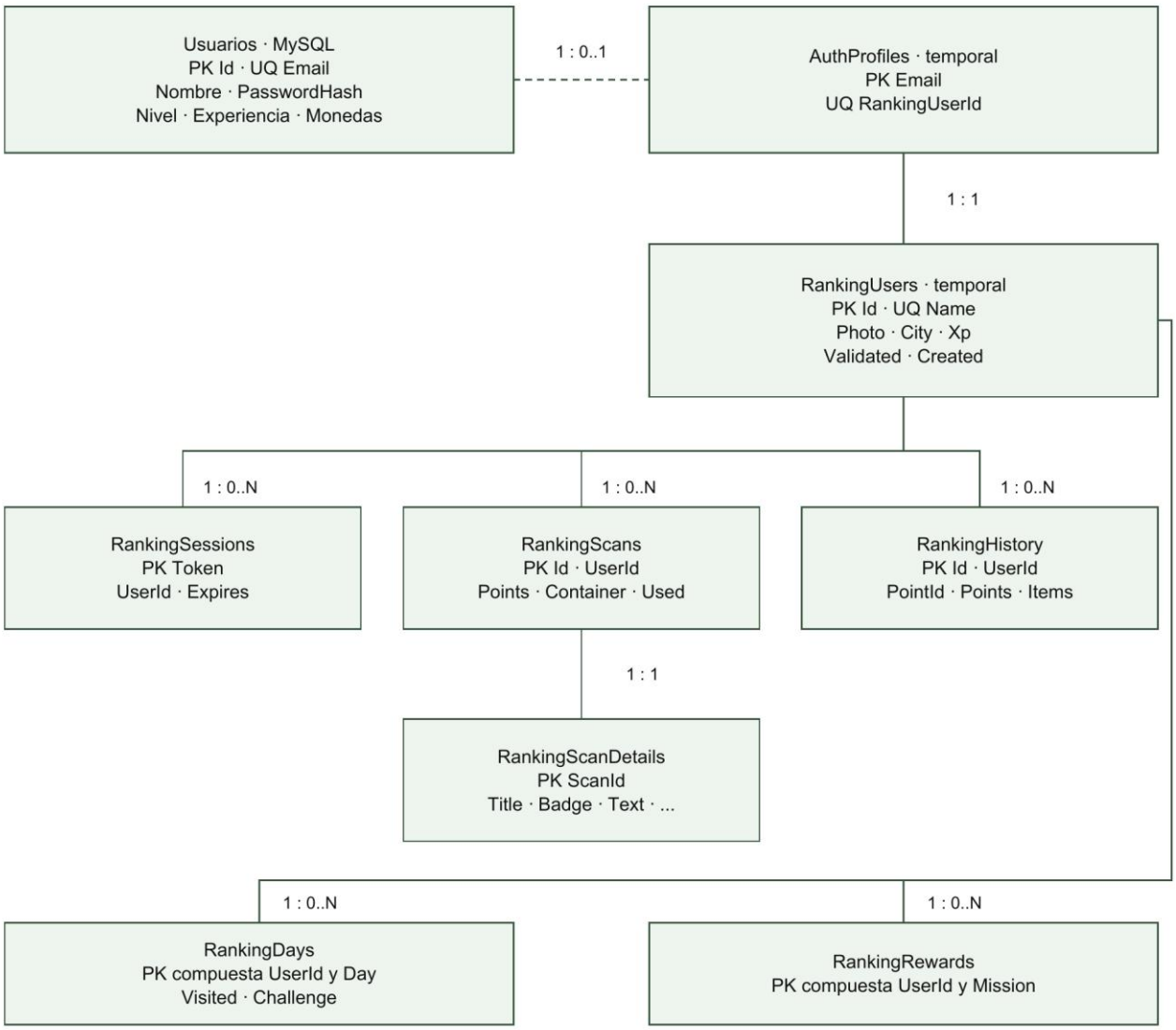
El navegador obtiene la ubicación con permiso del dispositivo.

Módulo y archivos	**Responsabilidad y dependencia**
Cuentas AuthController · AuthService UsuarioRepository	Registrar, validar y recuperar cuentas desde MySQL. No administra recuperación de contraseña.
Análisis WasteAnalysisController WasteAnalysisService	Enviar foto a Gemini; devolver recomendación y registrar un pendiente si corresponde.
Mapa RecyclingPointsController OfficialRecyclingPointRepository	Leer catálogo incluido y mostrar puntos. El navegador aporta ubicación.
Juego RankingController · RankingStore RewardRules · rewards.json	Sesiones, XP, validación y premios en SQLite en memoria; reglas desde JSON.
Presentación y preferencias ecoquest-account.js · ecoquest-state.js ecoquest-motion.js	Navegación y sincronización del perfil. Movimiento de animaciones como preferencia local.

HTML, CSS y JavaScript permiten una interfaz de navegador. ASP.NET Core recibe JSON y aplica reglas en el servidor. MySql.Data conecta las cuentas; Microsoft.Data.Sqlite mantiene el juego temporal. Swagger/OpenAPI describe las rutas en desarrollo. Los catálogos incluidos reducen la dependencia de consultas externas para obtener los puntos.

4 Modelo de datos y relaciones

El DER muestra las entidades implementadas y sus cardinalidades. PK identifica una clave primaria; UQ, una restricción de unicidad. Solo Usuarios se almacena en MySQL. Las demás tablas viven en SQLite en memoria.



Las líneas expresan relaciones lógicas utilizadas por el código, no claves foráneas declaradas en SQL. Usuarios.Email se vincula a AuthProfiles.Email; AuthProfiles.RankingUserId apunta a RankingUsers.Id. Una cuenta puede no tener perfil temporal hasta registrarse o iniciar sesión.

RankingUsers se relaciona por UserId con sesiones, escaneos, días, recompensas e historial. RankingScanDetails.ScanId identifica el detalle de un escaneo. La referencia a PointId corresponde al catálogo JSON, no a una tabla MySQL. La vinculación con la cuenta y todo el juego temporal desaparecen al reiniciar la API.

4 Diccionario de datos

4.1 Entradas y salidas

Entradas: nombre, email o nombre de acceso, contraseña, imagen en Data URL, identificadores de escaneos y de punto, latitud, longitud y precisión. Salidas: token, perfil, recomendación, catálogo de puntos, pendientes, XP, historial y ranking. Las credenciales no se devuelven en las respuestas.

4.2 Usuarios persistentes en MySQL

Campo	Tipo y regla
Id	INT, PK, AUTO_INCREMENT, no nulo
Nombre	VARCHAR(24), no nulo; validación de 3 a 24 caracteres
Email	VARCHAR(254), no nulo, índice único
PasswordHash	VARCHAR(255), no nulo; hash generado por PasswordHasher
Nivel / Experiencia / Monedas	INT, no nulos; valores iniciales 1 / 0 / 0

Los campos Nivel, Experiencia y Monedas de Usuarios no se actualizan con el juego actual. El XP activo está en RankingUsers. Al crear un perfil temporal se toma Experiencia como valor inicial; las cuentas nuevas comienzan en cero.

4.3 Tablas temporales en SQLite

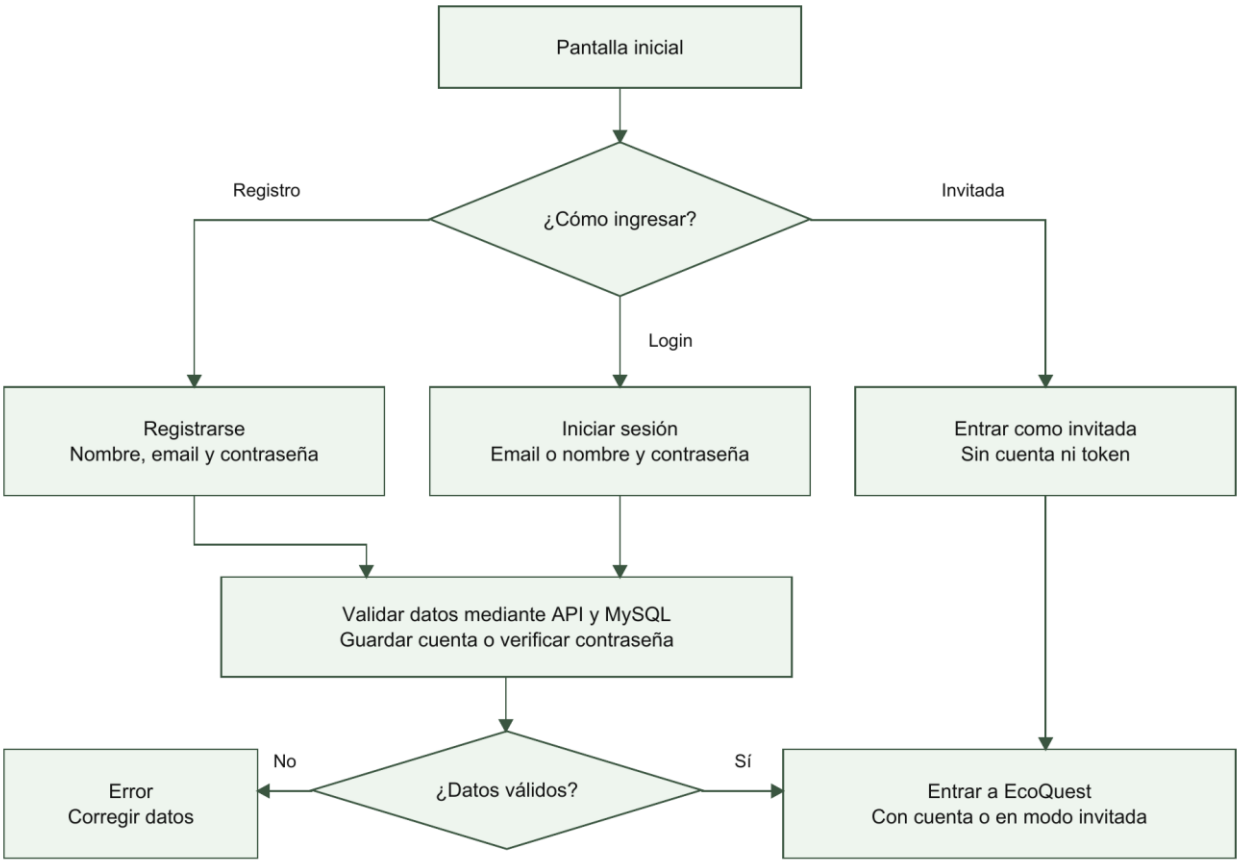
Tabla	Campos y tipos principales
AuthProfiles	Email TEXT PK; RankingUserId TEXT único.
RankingUsers	Id TEXT PK; Name TEXT único; Photo, City, Created TEXT; Xp y Validated INTEGER.
RankingSessions	Token TEXT PK (hash del token); UserId y Expires TEXT.
RankingScans	Id TEXT PK; UserId y Container TEXT; Points y Used INTEGER. Used inicia en 0.
RankingScanDetails	ScanId TEXT PK; Title, Badge, Text, WasteType, Confidence, ScannedAt TEXT; CanUseGreenContainer INTEGER.
RankingHistory	Id INTEGER PK autoincremental; UserId, PointId, PointName, Items y RecycledAt TEXT; Points INTEGER. Items contiene JSON.
RankingDays	PK compuesta (UserId TEXT, Day TEXT); Visited y Challenge INTEGER, inicialmente 0.
RankingRewards	PK compuesta (UserId TEXT, Mission TEXT); evita duplicar cada premio.

Los puntos y sus materiales, así como las reglas de misiones y rachas, se cargan desde archivos JSON. No requieren crear tablas adicionales en ecoquest.

5 Procesos de acceso

5.1 Registro e inicio de sesión

La pantalla inicial permite crear una cuenta, iniciar sesión o entrar como invitada. El registro valida los datos y crea Usuarios. El acceso verifica el hash de la contraseña. En ambos casos exitosos se vincula un perfil temporal y se devuelve un token.

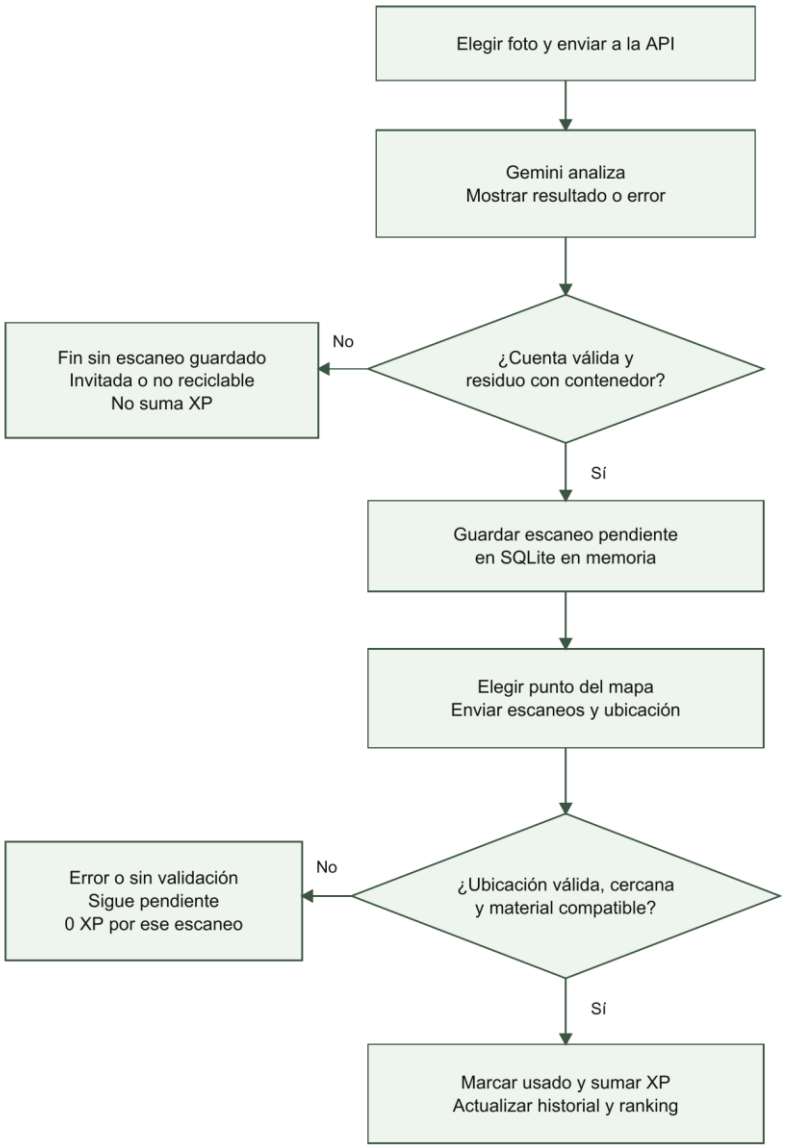


Si los datos son inválidos, se muestra un error y la persona puede corregirlos. El acceso como invitada no crea una cuenta ni una sesión de servidor. La invitada puede navegar y obtener una recomendación de IA si el servicio está configurado, pero no guarda escaneos ni recibe XP del servidor.

5.2 Cierre de sesión y reinicio

Cerrar sesión elimina el token activo del almacén temporal. No elimina la cuenta MySQL ni todo el perfil de juego mientras la API continúa abierta. Reiniciar la API sí elimina los perfiles, tokens y avances temporales; será necesario volver a iniciar sesión.

5 Proceso de análisis y validación

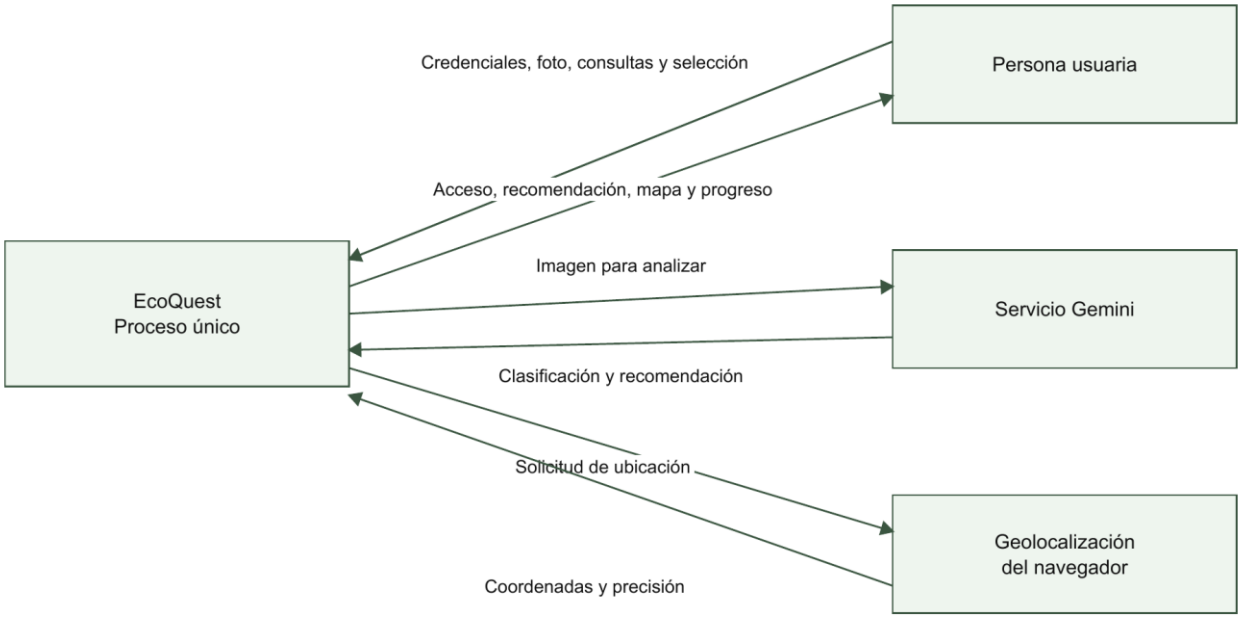


El pendiente se registra al analizar una imagen reciclable con cuenta válida, antes de validar el punto. El texto “+10” del resultado indica los puntos posibles; la foto sola no acredita reciclaje ni otorga ese XP.

Para verificar se aceptan de 1 a 20 IDs, un punto existente, coordenadas finitas, precisión mayor que 0 y de hasta 150 metros, y distancia de hasta 150 metros al punto. Se procesa cada ID distinto que pertenezca al usuario, esté pendiente y sea compatible. Los demás no suman puntos; una solicitud puede terminar con 0 XP o validar solo parte de sus escaneos.

5 Diagrama de contexto

EcoQuest se representa como un único proceso que intercambia datos con la persona usuaria, Gemini y la geolocalización del navegador. Las cuentas MySQL, el estado temporal y los archivos JSON son internos al sistema.

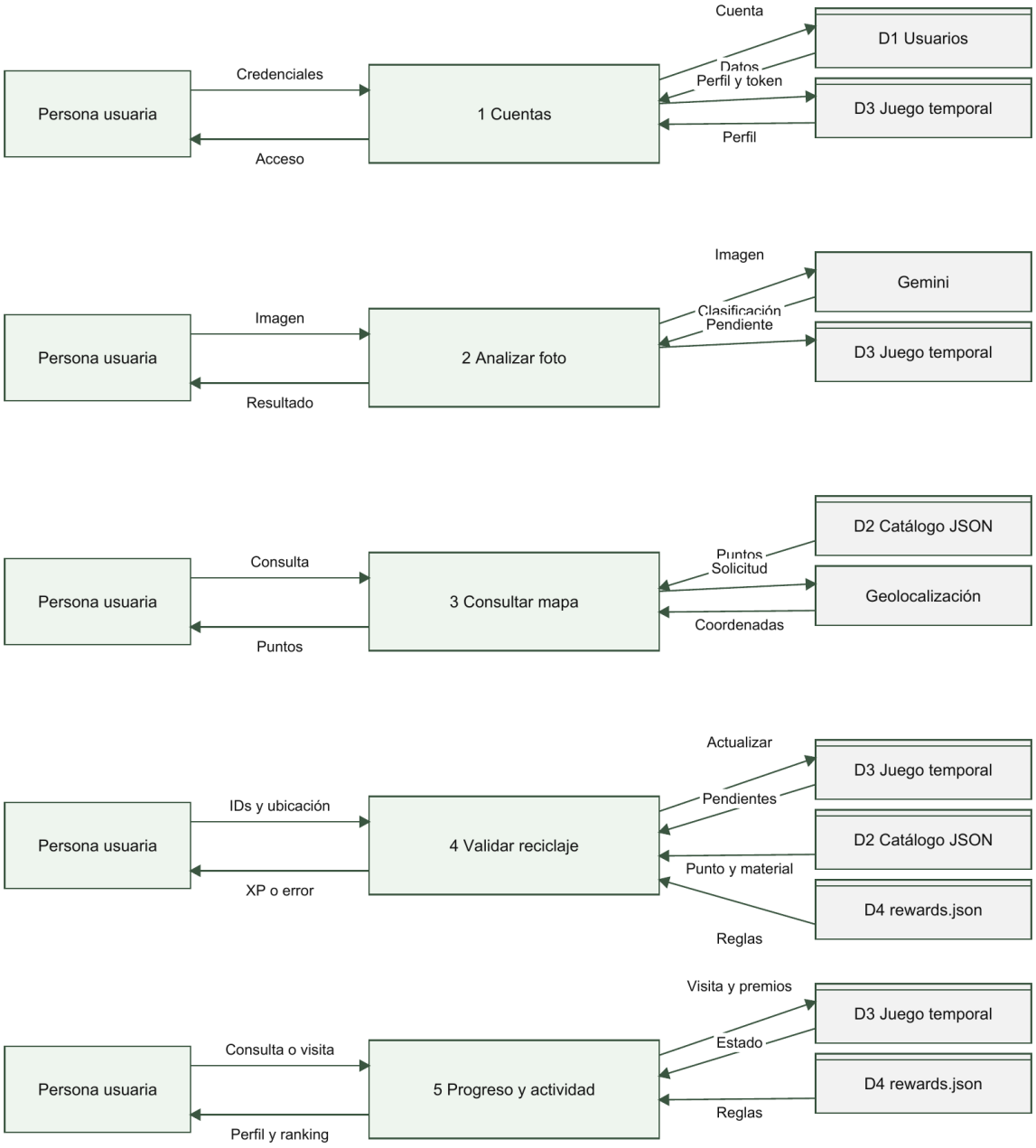


5.3 Intercambios con el exterior

Entidad	Envía al sistema	Recibe del sistema
Persona usuaria	Credenciales, foto, consultas y selección del punto.	Resultado de acceso, recomendación, puntos, progreso y errores.
Gemini	Clasificación y recomendación, o fallo del servicio.	Imagen y solicitud de análisis.
Geolocalización	Coordenadas y precisión, o permiso denegado.	Solicitud de ubicación desde el navegador.

No se incluyen un administrador ni un servicio de notificaciones porque no son actores implementados. La geolocalización se obtiene en el navegador y la API recibe las coordenadas para comprobar distancia y precisión.

5 Diagrama de flujo de datos de nivel 1



D1 es MySQL; D3 es SQLite en memoria; D2 y D4 son archivos JSON de lectura. La persona y los almacenes se repiten para evitar cruces: cada aparición representa la misma entidad o almacén. Los procesos intercambian estado a través de D3. La ubicación se solicita desde el navegador y se envía con los IDs al verificar. En D3 solo se registra un pendiente si hay cuenta y residuo válido.

6 Pantallas y reglas del juego

6.1 Correspondencia con procesos y datos

Opción del prototipo	Proceso y datos utilizados
Registrarse / Iniciar sesión	Proceso 1; Usuarios, AuthProfiles y RankingSessions.
Entrar como invitada	Acceso sin cuenta; no crea datos de juego en el servidor.
Inicio / Calendario	Proceso 5; perfil, RankingDays y premios de racha. Permite recorrer meses y acceder al escaneo.
Escanear	Proceso 2; Gemini y RankingScans/RankingScanDetails cuando corresponde.
Mapa / validar residuos	Procesos 3 y 4; catálogo JSON, pendientes y ubicación. Actualiza historial y XP.
Misiones y logros	Proceso 5; metas de rewards.json, Validated y RankingRewards. No existe una tabla persistente Misiones.
Ranking	Proceso 5; RankingUsers ordenado por XP, luego creación e ID. Es de CABA, sin filtro implementado por barrio.
Configuración	Cuenta y cierre de sesión; preferencia local de movimiento. No requiere una entidad MySQL adicional.
Mascota	Pantalla animada con aviso “Próximamente”. No implementa cuidados, compras ni accesorios.

6.2 Reglas verificadas en rewards.json y el servidor

La primera visita de cada día otorga 5 XP. Cada residuo validado aporta 10 XP. El primer reciclaje válido del día permite un bono de desafío de 15 XP. El nivel se calcula como 1 + parte entera de XP / 100. La fecha se calcula en la zona de Buenos Aires.

La racha cuenta días consecutivos de visita; si hoy todavía no se visitó, se cuenta desde ayer. Los premios son 25, 50, 100, 150 y 250 XP al llegar a 3, 7, 14, 20 y 30 días. Después se agregan premios de 250 XP cada 30 días adicionales. Cada premio se registra una sola vez por perfil temporal.

Las misiones dependen del total Validated: por ejemplo, Novato verde requiere 4 residuos y otorga 50 XP; Guardia del planeta requiere 12 y otorga 120 XP. Las metas restantes se leen del mismo archivo de reglas. El servidor evita volver a premiar un mismo escaneo marcado como usado.

7 Especificación de la API

Origen local: http://localhost:5228. Los cuerpos de solicitudes y respuestas usan JSON, incluida la foto codificada como Data URL. En desarrollo se puede consultar /swagger. Las rutas protegidas reciben Authorization: Bearer seguido del token devuelto por registro o login.

7.1 Cuentas y análisis

Método y ruta	Entrada y propósito	Respuesta
POST /api/auth/registro	nombre, email, contraseña: string obligatorios. Crea una cuenta.	200: mensaje, token y profile. 400: datos o email no disponibles; 409: conflicto de unicidad; 429: límite de solicitudes.
POST /api/auth/login	email: string (email o nombre); contraseña: string. Valida la cuenta.	200: mensaje, token y profile. 401: credenciales incorrectas; 429: límite.
POST /api/ranking/logout	Encabezado con token. Elimina esa sesión si está presente.	204 sin cuerpo.
POST /api/ia/analizar	imageDataUrl: string obligatorio. Token opcional para registrar pendiente.	200: clasificación; 400: imagen inválida; 401: token enviado pero inválido; 503: falta clave; 502: fallo de Gemini.

Resultado de análisis: title, badge, points, container, text, wasteType y confidence son string; isWaste y canUseGreenContainer son boolean; scanId es string o null. Los points de la recomendación no son una actualización del XP del perfil.

7.2 Mapa y disponibilidad

Método y ruta	Entrada y propósito	Respuesta
GET /api/contenedores GET /api/puntos-verdes	Sin parámetros. Ambas rutas invocan el mismo servicio de catálogo.	200: colección de puntos; incluye identificación, ubicación y materiales.
GET /api/health	Sin parámetros. Comprueba que responde la API.	200: {"status":"ok"}. No prueba por sí sola Gemini ni todo el recorrido de usuario.

Los errores de infraestructura pueden impedir el arranque: la conexión MySQL se comprueba al construir el repositorio de cuentas. La respuesta de health solo existe después de iniciar correctamente la aplicación.

7 API de progreso y ranking

Método y ruta	Entrada y propósito	Respuesta
GET /api/ranking	limit: int opcional, 10 por defecto; se limita a 1..100. Público.	200: city:string y users:array con id, name, xp, rank y photoUrl.
GET /api/ranking/me	Token obligatorio. Consulta perfil.	200: perfil; 401: sin sesión válida.
GET /api/ranking/progress	Token obligatorio. Consulta avance.	200: profile, pendingScans e history; 401 sin sesión.
POST /api/ranking/activity	Token obligatorio; sin cuerpo. Registra visita diaria.	200: awardedPoints:int y profile; 401 sin sesión.
POST /api/ranking/verify	Token; scanIds:string[]; pointId:string; lat, lng y accuracy:number.	200: awardedPoints y profile. 400: datos de ubicación o punto inválidos; 401 sin sesión.
PUT /api/ranking/photo	Token; cuerpo JSON string con Data URL JPEG. Cadena vacía elimina foto.	200: profile; 400: foto inválida; 401 sin sesión.
GET /api/ranking/photo/{id}	id:string del perfil temporal. Sin token.	200: bytes image/jpeg; 404: sin foto.

7.3 Contrato de verificación

scanIds identifica entre 1 y 20 pendientes; pointId identifica un punto del catálogo; lat y lng son coordenadas en grados, y accuracy es la precisión reportada en metros. Un contenedor incompatible o un escaneo ajeno, inexistente o ya usado no recibe XP, aunque la respuesta global pueda ser 200.

```
{
  "scanIds": ["ID_DEVUELTO_POR_EL_ANALISIS"],
  "pointId": "ID_DEL_CATALOGO",
  "lat": -34.60, "lng": -58.40, "accuracy": 20
}
```

El ejemplo describe la forma del JSON; los IDs y las coordenadas deben provenir del recorrido real. No es una muestra lista para validar un punto concreto.

7.4 Perfil devuelto

Incluye id, name, xp, validated, scans, city, photoUrl, completedMissionIds, level y activity. La actividad contiene visitas, desafíos, racha y próximo premio. pendingScans devuelve hasta 20 pendientes. El token se almacena como hash y vence a los 30 días o antes si la API se reinicia.

8 Pruebas y conclusiones

8.1 Estado de verificación

Comprobación	Resultado de esta revisión
Compilar backend con SDK 10.0.302	Correcta: 0 errores y 0 advertencias.
Concordancia de rutas, reglas y datos	Revisada por lectura de controladores, repositorio, RankingStore, frontend y rewards.json.

8.2 Prueba de aceptación después de instalar

Crear una cuenta de prueba, ingresar, cerrar y volver a ingresar. Detener y reiniciar la API: la cuenta debe seguir disponible en MySQL y el juego temporal debe reiniciarse. Probar también la entrada como invitada.

Con Gemini configurado, analizar una foto reciclable: con cuenta debe aparecer un pendiente y como invitada no debe guardarse. Probar una imagen inválida, permiso de ubicación denegado, punto incompatible y distancia mayor a 150 m. En un punto compatible y cercano, validar una vez y comprobar que repetir el mismo ID no otorga otro premio. Consultar XP, historial, misiones, ranking y vista móvil.

Estos son casos de prueba para ejecutar; no se presentan como resultados aprobados. Los datos de cuentas de ensayo y de juego son de demostración. La IA no tiene un modo simulado incorporado y el ranking puede empezar vacío.

8.3 Conclusiones y dificultades

EcoQuest conecta pantallas web, API, cuentas persistentes y catálogo de reciclaje. El desarrollo permitió separar responsabilidades y distinguir entradas, salidas y almacenamiento. Las principales limitaciones son la disponibilidad de Gemini, la precisión de ubicación y la pérdida de progreso al reiniciar el servidor. Una versión futura puede persistir el juego, completar la mascota y desplegar el prototipo con HTTPS.

8.4 Material de entrega

Esta documentación se presenta en un único PDF. El código se obtiene desde https://github.com/criseri23/ecoquest2 y se ejecuta con el procedimiento de la sección 2. El acceso http://localhost:5228/pages/click.html funciona únicamente en el equipo que inició la API.